

Programación reactiva en el frontend

Conclusión

Sí, el frontend utiliza programación reactiva, pero de forma **moderada y estándar** para Angular. La reactividad se implementa mediante **RxJS (Observables)**, que es el mecanismo reactivo nativo de Angular para manejar flujos asíncronos.

Sin embargo, no es una reactividad "extrema": no se usa NgRx (Redux reactivo), ni Signals de Angular de forma avanzada, ni streams complejos con operadores reactivos elaborados. Es la reactividad típica de una aplicación Angular: **Observables para HTTP y suscripciones en componentes**.

Evidencia de programación reactiva en el código

1. RxJS como dependencia (package.json)

```
"dependencies": {  
  "@angular/core": "~17.3.12",  
  "rxjs": "^7.8.1",  
  "zone.js": "~0.14.0"  
}
```

- rxjs es la librería de programación reactiva de JavaScript (Observables, Subjects, operadores).
- zone.js es el mecanismo de detección de cambios reactivo de Angular.

2. Servicios que devuelven Observable (product.service.ts)

```
import { Observable } from 'rxjs';  
  
getProducts(category?: string): Observable<Product[]> {  
  let params = new HttpParams();  
  if (category?.trim()) {  
    params = params.set('category', category.trim());  
  }  
  return this.http.get<Product[]>(this.apiUrl, { params });  
}  
  
getProduct(id: number): Observable<Product> {  
  return this.http.get<Product>(`${this.apiUrl}/${id}`);  
}  
  
createProduct(payload: Product): Observable<Product> {  
  return this.http.post<Product>(this.apiUrl, payload);  
}
```

- Todos los métodos del servicio devuelven `Observable<T>`, el tipo reactivo de RxJS.
- `HttpClient` de Angular es intrínsecamente reactivo: las peticiones HTTP se modelan como streams.

3. Suscripciones en componentes (product-list.component.ts)

```
loadProducts(): void {  
  this.loading = true;  
  this.error = '';  
  this.productService.getProducts(this.categoryFilter).subscribe({  
    next: data => {  
      this.products = data;  
      this.loading = false;  
    },  
    error: () => {  
      this.error = 'No se pudieron cargar los productos';  
      this.loading = false;  
    }  
  });  
}
```

- El componente se **suscribe** al Observable con `.subscribe()`.
- Se manejan los tres canales reactivos: `next` (datos), `error` (fallo) y `complete` (finalización).
- El flujo de datos es **asíncrono y no bloqueante**: la UI no se congela mientras espera la respuesta HTTP.

4. Patrón reactivo en ambos frontends

Tanto `frontend/` como `frontendV2/` siguen exactamente el mismo patrón reactivo:

Archivo	Patrón reactivo
<code>product.service.ts</code>	Devuelve Observable en todos los métodos
<code>product-list.component.ts</code>	Se suscribe con <code>subscribe</code> (<code>next</code> , <code>error</code>)
<code>product-form.component.ts</code>	Se suscribe para crear/actualizar productos
<code>product-detail.component.ts</code>	Se suscribe para obtener un producto por id

Evidencia de que NO es reactividad avanzada

1. No se usan operadores RxJS complejos

```
// Lo que se usa: suscripción directa
this.productService.getProducts(this.categoryFilter).subscribe({ ... });

// Lo que NO se usa: operadores reactivos avanzados
// .pipe(map(...), switchMap(...), debounceTime(...), catchError(...), takeUntil(...))
```

- No hay `pipe()`, `map()`, `switchMap()`, `debounceTime()`, `catchError()`, `takeUntil()` ni otros operadores.
- Las suscripciones son directas y simples.

2. No se gestiona la cancelación de suscripciones

```
ngOnInit(): void {
  this.loadProducts();
}
```

- No se usa `takeUntilDestroyed()`, `unsubscribe()` en `ngOnDestroy()`, ni `AsyncPipe` en las plantillas.
- Esto puede causar **memory leaks** si el componente se destruye mientras la petición está en curso.

3. No se usa NgRx ni Signals avanzados

- No hay `@ngrx/store`, `@ngrx/effects` ni `@ngrx/entity` en `package.json`.
- No se usa `signal()` ni `computed()` de Angular Signals.
- El estado de los componentes es mutable y local (`products: Product[] = []`).

4. No hay streams combinados ni reactividad entre componentes

- No se usan `Subject`, `BehaviorSubject` ni `EventEmitter` para comunicación reactiva entre componentes.
- No hay `combineLatest`, `forkJoin`, `merge` ni `concat`.
- La reactividad se limita a peticiones HTTP individuales.

Resumen comparativo

Característica	¿Usada en el frontend?
Observables de RxJS	Sí
HttpClient reactivo	Sí

Característica	¿Usada en el frontend?
Suscripciones con subscribe	Sí
Operadores RxJS (pipe, map, switchMap)	No
Cancelación de suscripciones	No
NgRx (Redux reactivo)	No
Angular Signals	No
Subjects / BehaviorSubject	No
AsyncPipe en plantillas	No

Conclusión final

El frontend **sí utiliza programación reactiva** en el sentido de que toda la comunicación con el backend se modela con **Observables de RxJS**, que es el estándar reactivo de Angular. Las peticiones HTTP son asíncronas y no bloqueantes, y los componentes reaccionan a los datos cuando llegan.

Sin embargo, es una **reactividad básica**: no se aprovechan los operadores RxJS, no se gestiona la cancelación de suscripciones, y no se usan patrones reactivos avanzados como NgRx o Signals. Es el nivel de reactividad que Angular ofrece "de fábrica" sin configuración adicional.

Este enfoque es el más común en aplicaciones Angular de tamaño pequeño/mediano: **reactividad pragmática** con Observables para HTTP, sin la complejidad de un estado global reactivo.